```
/**
 * Motor de WhatsApp — Keller IA
 *
 * O que isso faz:
 *  - Conecta num número de WhatsApp (o SEU número, escaneando um QR Code na pági
 *  - Expõe um endpoint POST /enviar que o Painel de Leads chama pra mandar mensa
 *    automaticamente pro cliente, sem precisar abrir o WhatsApp manualmente
 *  - (Opcional) Se a variável GROQ_API_KEY estiver configurada, também responde
 *    sozinho quem mandar mensagem, usando os dados da empresa em /empresas
 *
 * Importante: usa a biblioteca Baileys, que fala com o WhatsApp de forma NÃO ofi
 * (não é a API paga do Meta). É de graça, mas carrega algum risco de o número se
 * sinalizado se o volume de mensagens ficar muito alto ou repetitivo.
 *
 * ATENÇÃO SOBRE O PLANO GRATUITO DO RENDER:
 * O disco é apagado toda vez que o serviço reinicia (o que acontece depois de um
 * tempo sem uso, ou a cada novo deploy). Isso significa que a sessão do WhatsApp
 * (o "login") pode precisar ser refeita de vez em quando — vai aparecer um novo
 * code na página inicial quando isso acontecer. Se isso incomodar, o plano pago
 * Render com disco persistente resolve de vez (fica sempre conectado).
 */

require("dotenv").config();
const express = require("express");
const fs = require("fs");
const path = require("path");
const pino = require("pino");
const QRCode = require("qrcode");
const {
  default: makeWASocket,
  useMultiFileAuthState,
  DisconnectReason,
  fetchLatestBaileysVersion,
} = require("@whiskeysockets/baileys");

const EMPRESAS_DIR = path.join(__dirname, "empresas");
const AUTH_DIR = path.join(__dirname, "auth_info");
const PORT = process.env.PORT || 3000;
const API_KEY = process.env.API_KEY || ""; // chave que o painel de leads usa pra

// IA opcional de auto-resposta — só ativa se a chave estiver configurada.
let groq = null;
if (process.env.GROQ_API_KEY) {
  try {
```

```
    const Groq = require("groq-sdk");
    groq = new Groq({ apiKey: process.env.GROQ_API_KEY });
    console.log("🤖 Auto-resposta por IA ativada (GROQ_API_KEY encontrada).");
  } catch (erro) {
    console.error("Não consegui iniciar a IA de auto-resposta (seguindo sem ela):
  }
} else {
  console.log("ℹ️  GROQ_API_KEY não configurada — auto-resposta por IA desativada
}

// Memória de conversa por número (em RAM — reseta se o processo reiniciar).
const historico = new Map();
const HISTORICO_MAX_MENSAGENS = 10;

let sock = null;
let ultimoQrDataUrl = null;
let statusConexao = "iniciando"; // iniciando | aguardando_qr | conectado | caiu

function carregarEmpresa(numeroCliente) {
  const arquivo = path.join(EMPRESAS_DIR, `${numeroCliente}.json`);
  if (fs.existsSync(arquivo)) {
    return JSON.parse(fs.readFileSync(arquivo, "utf-8"));
  }
  const exemplo = path.join(EMPRESAS_DIR, "exemplo.json");
  if (fs.existsSync(exemplo)) {
    return JSON.parse(fs.readFileSync(exemplo, "utf-8"));
  }
  return null;
}

function montarPrompt(empresa) {
  return `Você é a secretária virtual de "${empresa.nome_empresa}", um(a) ${empre

REGRAS IMPORTANTES:
- Responda SOMENTE com base nas informações abaixo. NUNCA invente preço, horário,
- Se perguntarem algo que você não tem informação pra responder, diga educadament
- Seja breve, simpática e direta — mensagens curtas, como se estivesse no WhatsAp
- Nunca se apresente como "inteligência artificial" a menos que perguntem diretam

DADOS DA EMPRESA:
- Endereço: ${empresa.endereco}
- Horário de funcionamento: ${empresa.horario_funcionamento}
- Serviços e preços: ${(empresa.servicos || []).map(s => `${s.nome} (${s.preco})`
- Como agendar: ${empresa.forma_agendamento}
- Quando chamar um humano: ${empresa.quando_chamar_humano}
`;
}
```

```
async function responderComIA(empresa, numeroConversa, textoRecebido) {
  const historicoConversa = historico.get(numeroConversa) || [];
  const mensagens = [
    { role: "system", content: montarPrompt(empresa) },
    ...historicoConversa,
    { role: "user", content: textoRecebido },
  ];

  const resposta = await groq.chat.completions.create({
    model: "llama-3.3-70b-versatile",
    messages: mensagens,
    temperature: 0.4,
    max_tokens: 300,
  });

  const textoResposta = resposta.choices[0]?.message?.content?.trim()
    || "Desculpa, deu um probleminha aqui. Já já alguém da equipe te responde!";

  const novoHistorico = [
    ...historicoConversa,
    { role: "user", content: textoRecebido },
    { role: "assistant", content: textoResposta },
  ].slice(-HISTORICO_MAX_MENSAGENS);
  historico.set(numeroConversa, novoHistorico);

  return textoResposta;
}

// Transforma "5514988324456" ou "14988324456" no formato que o Baileys espera (J
function paraJid(numero) {
  let digitos = String(numero).replace(/\D/g, "");
  if (digitos.length <= 11) digitos = "55" + digitos; // assume Brasil se vier se
  return `${digitos}@s.whatsapp.net`;
}

// ---------- Envio automático diário dos leads do Painel (Firestore) ----------
// Usa a API REST do Firestore direto (as regras do banco já permitem leitura/esc
// pública nessas coleções, então não precisa de credencial de servidor).
const FIRESTORE_PROJECT = "escritorio-c3acb";
const FIRESTORE_API_KEY = "AIzaSyBZyZ4Dqs0BI5VSsm3HB2TNGpk7Z7330CE";
const FIRESTORE_BASE = `https://firestore.googleapis.com/v1/projects/${FIRESTORE_
const LIMITE_ENVIOS_POR_DIA = 20;
const ENVIOS_POR_EXECUCAO = 2; // quantos leads tenta mandar a cada vez que o /au

function fsValorParaJs(v) {
  if (v == null) return null;
```

```
  if ("stringValue" in v) return v.stringValue;
  if ("integerValue" in v) return parseInt(v.integerValue, 10);
  if ("doubleValue" in v) return v.doubleValue;
  if ("booleanValue" in v) return v.booleanValue;
  if ("nullValue" in v) return null;
  if ("mapValue" in v) return fsCamposParaJs(v.mapValue.fields || {});
  if ("arrayValue" in v) return (v.arrayValue.values || []).map(fsValorParaJs);
  return null;
}
function fsCamposParaJs(fields) {
  const obj = {};
  for (const k in fields) obj[k] = fsValorParaJs(fields[k]);
  return obj;
}
function jsParaFsValor(v) {
  if (typeof v === "string") return { stringValue: v };
  if (typeof v === "number") return Number.isInteger(v) ? { integerValue: String(
  if (typeof v === "boolean") return { booleanValue: v };
  return { nullValue: null };
}

async function firestoreGetDoc(caminho) {
  const resp = await fetch(`${FIRESTORE_BASE}/${caminho}?key=${FIRESTORE_API_KEY}
  if (resp.status === 404) return null;
  if (!resp.ok) throw new Error(`Firestore GET ${caminho} falhou: ${resp.status}`
  const data = await resp.json();
  return fsCamposParaJs(data.fields || {});
}

async function firestoreListarColecao(nome) {
  const resp = await fetch(`${FIRESTORE_BASE}/${nome}?pageSize=300&key=${FIRESTOR
  if (!resp.ok) throw new Error(`Firestore LIST ${nome} falhou: ${resp.status}`);
  const data = await resp.json();
  return (data.documents || []).map((doc) => {
    const id = doc.name.split("/").pop();
    return { id, ...fsCamposParaJs(doc.fields || {}) };
  });
}

async function firestoreSetCampos(caminho, campos) {
  const mask = Object.keys(campos).map((k) => `updateMask.fieldPaths=${encodeURIC
  const fields = {};
  for (const k in campos) fields[k] = jsParaFsValor(campos[k]);
  const resp = await fetch(`${FIRESTORE_BASE}/${caminho}?${mask}&key=${FIRESTORE_
    method: "PATCH",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ fields }),
```

```
  });
  if (!resp.ok) throw new Error(`Firestore PATCH ${caminho} falhou: ${resp.status
}

// Mesma lógica de mensagem padrão usada no Painel de Leads (netlify-site/index.h
const ABERTURAS = [
  "Oi! Tudo bem? Me chamo Keller",
  "Olá! Aqui é o Keller",
  "Oi, bom dia! Me chamo Keller",
  "Olá! Tudo certo? Aqui é o Keller",
];
const FECHOS = [
  "Posso te mostrar rapidinho como funciona?",
  "Topa dar uma olhada?",
  "Faz sentido pra vocês? Posso te explicar melhor.",
  "Quer que eu te mande mais detalhes?",
];
function hashIndex(str, mod) {
  let h = 0;
  for (let i = 0; i < str.length; i++) h = (h * 31 + str.charCodeAt(i)) >>> 0;
  return h % mod;
}
function mensagemPadrao(lead, empresaConfig) {
  const empresaLead = lead.empresa || "tudo bem";
  const seed = String(lead.id || empresaLead || "");
  const abertura = ABERTURAS[hashIndex(seed, ABERTURAS.length)];
  const fecho = FECHOS[hashIndex(seed + "x", FECHOS.length)];
  let msg = "";
  if (empresaConfig && empresaConfig.nome) {
    msg += `${abertura}, da ${empresaConfig.nome}. `;
    if (empresaConfig.nicho) {
      msg += `Vi o ${empresaLead} aqui${lead.cidade ? " em " + lead.cidade : ""}
    } else {
      msg += `Vi o ${empresaLead} aqui. `;
    }
    if (empresaConfig.promocao) {
      msg += `Essa semana estamos com uma condição especial: ${empresaConfig.prom
    }
    msg += fecho;
    return msg;
  }
  const segmento = (lead.segmento || "").toLowerCase();
  msg += `${abertura}, da Keller IA. `;
  if (segmento) {
    msg += `Vi o ${empresaLead} aqui e ajudo negócios de ${segmento} a responder
  } else {
    msg += `Vi o ${empresaLead} aqui e ajudo empresas a terem um atendimento auto
```

```
  }
  msg += fecho;
  return msg;
}

function hojeData() {
  // Data no formato AAAA-MM-DD, fuso de São Paulo (UTC-3), só pra controlar o li
  const agora = new Date(Date.now() - 3 * 60 * 60 * 1000);
  return agora.toISOString().slice(0, 10);
}

async function iniciar() {
  const { state, saveCreds } = await useMultiFileAuthState(AUTH_DIR);

  let version;
  try {
    ({ version } = await fetchLatestBaileysVersion());
  } catch (erro) {
    console.error("Não consegui checar a versão mais recente do WhatsApp, seguind
    version = undefined;
  }

  sock = makeWASocket({
    version,
    auth: state,
    logger: pino({ level: "silent" }),
    printQRInTerminal: false,
  });

  sock.ev.on("creds.update", saveCreds);

  sock.ev.on("connection.update", async (update) => {
    const { connection, lastDisconnect, qr } = update;

    if (qr) {
      statusConexao = "aguardando_qr";
      try {
        ultimoQrDataUrl = await QRCode.toDataURL(qr);
        console.log("📱 Novo QR code gerado — abre a página inicial do serviço pr
      } catch (erro) {
        console.error("Erro ao gerar imagem do QR code:", erro);
      }
    }

    if (connection === "close") {
      statusConexao = "caiu";
      const deveReconectar =
```

```
        lastDisconnect?.error?.output?.statusCode !== DisconnectReason.loggedOut;
      console.log("Conexão com o WhatsApp caiu. Reconectar automaticamente?", dev
      if (deveReconectar) {
        setTimeout(() => iniciar().catch((e) => console.error("Erro ao reconectar
      }
    } else if (connection === "open") {
      statusConexao = "conectado";
      ultimoQrDataUrl = null;
      console.log("✅ Conectado ao WhatsApp!");
    }
  });

  sock.ev.on("messages.upsert", async ({ messages, type }) => {
    if (type !== "notify" || !groq) return; // sem IA configurada, não responde s

    for (const msg of messages) {
      if (!msg.message || msg.key.fromMe) continue;

      const numeroRemetente = msg.key.remoteJid;
      const texto =
        msg.message.conversation ||
        msg.message.extendedTextMessage?.text ||
        "";
      if (!texto) continue;

      const numeroEmpresa = sock.user?.id?.split(":")[0];
      const empresa = carregarEmpresa(numeroEmpresa);
      if (!empresa) continue;

      console.log(`📩 Mensagem de ${numeroRemetente}: ${texto}`);
      try {
        const resposta = await responderComIA(empresa, numeroRemetente, texto);
        await sock.sendMessage(numeroRemetente, { text: resposta });
        console.log(`📤 Respondido automaticamente: ${resposta}`);
      } catch (erro) {
        console.error("Erro ao responder automaticamente:", erro);
      }
    }
  });
}

// ---------- Servidor HTTP ----------
const app = express();
app.use(express.json());

app.get("/", (req, res) => {
  if (statusConexao === "conectado") {
```

```
    res.send(`
      <html><body style="font-family:sans-serif;text-align:center;padding:60px">
        <h1>✅ Motor WhatsApp Keller IA conectado</h1>
        <p>Pronto pra enviar mensagens automaticamente.</p>
      </body></html>
    `);
  } else if (ultimoQrDataUrl) {
    res.send(`
      <html><body style="font-family:sans-serif;text-align:center;padding:40px">
        <h1>📱 Escaneie pra conectar seu WhatsApp</h1>
        <p>No celular: WhatsApp → Configurações → Aparelhos conectados → Conectar
        <img src="${ultimoQrDataUrl}" style="width:300px;height:300px" />
        <p style="color:#888">A página atualiza sozinha a cada 5 segundos.</p>
        <script>setTimeout(() => location.reload(), 5000);</script>
      </body></html>
    `);
  } else {
    res.send(`
      <html><body style="font-family:sans-serif;text-align:center;padding:60px">
        <h1>⏳ Gerando QR code...</h1>
        <p>Atualiza a página em alguns segundos.</p>
        <script>setTimeout(() => location.reload(), 4000);</script>
      </body></html>
    `);
  }
});

app.get("/status", (req, res) => {
  res.json({ status: statusConexao });
});

// O Painel de Leads chama este endpoint pra mandar mensagem automaticamente.
app.post("/enviar", async (req, res) => {
  if (API_KEY && req.headers["x-api-key"] !== API_KEY) {
    return res.status(401).json({ erro: "Chave de API inválida." });
  }
  if (statusConexao !== "conectado" || !sock) {
    return res.status(503).json({ erro: "WhatsApp ainda não está conectado. Escan
  }

  const { numero, mensagem } = req.body || {};
  if (!numero || !mensagem) {
    return res.status(400).json({ erro: "Informe 'numero' e 'mensagem' no corpo d
  }

  try {
    await sock.sendMessage(paraJid(numero), { text: mensagem });
```

```
    res.json({ sucesso: true });
  } catch (erro) {
    console.error("Erro ao enviar mensagem via /enviar:", erro);
    res.status(500).json({ erro: "Falha ao enviar mensagem.", detalhe: erro.messa
  }
});

// Roda sozinho (chamado por um agendamento) e manda mensagem pros leads novos do
// Painel, respeitando um limite diário pra não arriscar o número ser bloqueado.
app.post("/auto-enviar", async (req, res) => {
  if (API_KEY && req.headers["x-api-key"] !== API_KEY) {
    return res.status(401).json({ erro: "Chave de API inválida." });
  }
  if (statusConexao !== "conectado" || !sock) {
    return res.status(503).json({ erro: "WhatsApp ainda não está conectado." });
  }

  try {
    const hoje = hojeData();
    let contador = await firestoreGetDoc("config/envioAutomatico");
    let enviadosHoje = contador && contador.data === hoje ? (contador.enviadosHoj

    if (enviadosHoje >= LIMITE_ENVIOS_POR_DIA) {
      return res.json({ enviados: 0, motivo: "Limite diário já atingido", enviado
    }

    const empresaConfig = (await firestoreGetDoc("config/empresa")) || {};
    const leads = await firestoreListarColecao("leads");
    const elegiveis = leads.filter((l) => (l.status || "Novo") === "Novo" && l.wh

    const quantos = Math.min(ENVIOS_POR_EXECUCAO, LIMITE_ENVIOS_POR_DIA - enviado
    const enviadosNestaExecucao = [];

    for (let i = 0; i < quantos; i++) {
      const lead = elegiveis[i];
      const mensagem = mensagemPadrao(lead, empresaConfig);
      try {
        await sock.sendMessage(paraJid(lead.whatsapp), { text: mensagem });
        await firestoreSetCampos(`leads/${lead.id}`, { status: "Contatado" });
        enviadosHoje++;
        enviadosNestaExecucao.push(lead.empresa || lead.id);
        await firestoreSetCampos("config/envioAutomatico", { data: hoje, enviados
        if (i < quantos - 1) await new Promise((r) => setTimeout(r, 8000)); // pe
      } catch (erro) {
        console.error(`Erro ao enviar automático pro lead ${lead.id}:`, erro);
      }
    }
```

```
    res.json({ enviados: enviadosNestaExecucao.length, para: enviadosNestaExecuca
  } catch (erro) {
    console.error("Erro no envio automático:", erro);
    res.status(500).json({ erro: "Falha no envio automático.", detalhe: erro.mess
  }
});

app.listen(PORT, () => console.log(`Servidor HTTP ouvindo na porta ${PORT}`));

iniciar().catch((err) => console.error("Erro ao iniciar conexão com o WhatsApp:",
```